

Project Completion Report: FarmVidhya Backend POC1

I. Executive Summary

This report summarizes the successful creation and local deployment of the Proof-of-Concept (POC1) backend for the FarmVidhya Conversational AI platform. Following a robust, scalable microservices architecture, we have built four independent yet interconnected services to handle user authentication, billing logic, job queuing, and asynchronous ML processing. The system is now fully functional on your local machine, providing a solid foundation for future development and the integration of real AI models.

II. The Development Journey: A Summary of Our Process

Our journey was a textbook example of modern software development, involving structured setup, coding, and iterative debugging to achieve a stable system.

Phase 0: Environment Setup

- Python: The core programming language.
- PostgreSQL: The powerful relational database server.
- RabbitMQ: The industry-standard message broker for handling background tasks.

Phase 1: Project Scaffolding & Coding

- We laid out a clean, professional project structure with separate folders for each microservice.
- Populated Python files with initial code, defining database models, API schemas, and business logic.

Phase 2: Iterative Debugging & Launch

- Corrected Python import errors (relative → absolute imports).
- Installed missing packages (email-validator).
- Fixed database permission errors (InsufficientPrivilege).
- Solved configuration errors (environment variables).
- Fixed code typos (e.g., Depends(get_d)).
- Result: A stable, multi-service application running in four terminals.

III. Detailed Service Architecture Breakdown

A. User Service (user_service folder)

- Manages user identity, registration, and authentication.
- Tech: Python, FastAPI, PostgreSQL, SQLAlchemy.
- Key Dependencies: fastapi, uvicorn, sqlalchemy, psycopg2-binary, passlib[bcrypt], python-jose[cryptography], email-validator.
- Handles secure sign-up, login, and JWT token generation.

B. Billing Service (billing_service folder)

- Handles user credits and AI model usage costs.
- Tech: Python, FastAPI, PostgreSQL, SQLAlchemy.
- Key Dependencies: fastapi, uvicorn, sqlalchemy, psycopg2-binary.

- Creates default policies, manages balances, grants free credits on first use.

C. Queue Service (queue_service folder)

- Acts as Job Dispatcher, validates requests, and delegates tasks to RabbitMQ.
- Tech: Python, FastAPI, PostgreSQL, SQLAlchemy, RabbitMQ.
- Key Dependencies: fastapi, uvicorn, sqlalchemy, psycpg2-binary, pika, python-jose[cryptography].
- Creates job records, validates JWT tokens, publishes messages to RabbitMQ.

D. Orchestration Service (orchestration_service folder)

- Background worker for AI processing.
- Tech: Python, PostgreSQL, SQLAlchemy, RabbitMQ.
- Key Dependencies: sqlalchemy, psycpg2-binary, pika.
- Listens to RabbitMQ, updates job statuses, simulates ML tasks.

IV. The Complete End-to-End Workflow (Flowchart View)

- i Step 1: Authentication (User Service) – Sign-up and login, return JWT token.
- ii Step 2: Job Submission (Queue Service) – User submits job, receives job_id.
- iii Step 3: Hand-off (RabbitMQ) – Queue Service publishes job_id to RabbitMQ.
- iv Step 4: Processing (Orchestration Service) – Worker picks job, processes it, updates status to COMPLETED.
- v Step 5: Checking Result (Queue Service) – User checks /jobs/{job_id} until result is available.